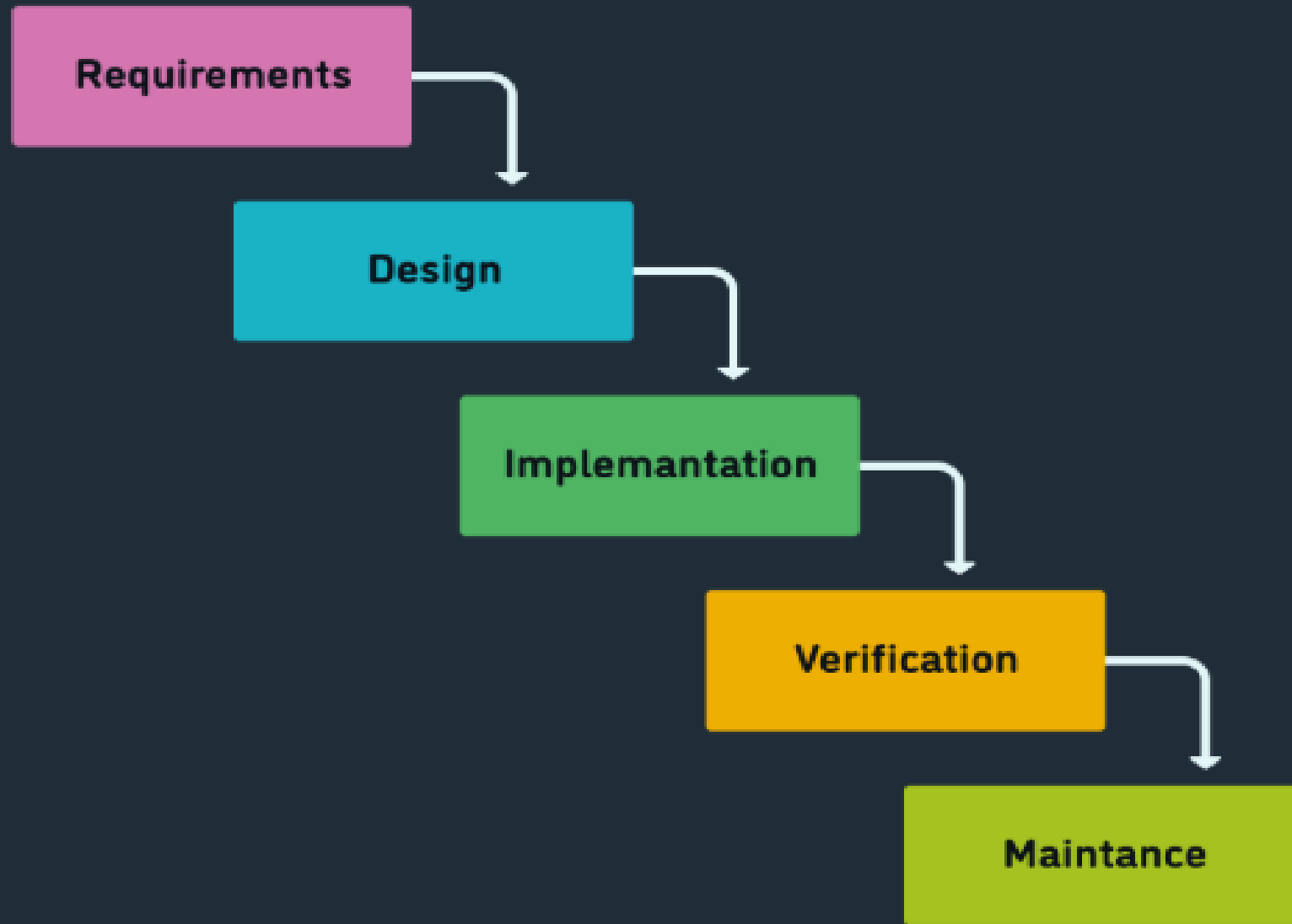


SYSTEM DESIGN

What is System Design?

- System design is the process of planning and structuring the architecture of a software system based on user requirements.
- It defines how different components of the system will work together to achieve the desired functionality efficiently.





Why is System Design Important



Scalability



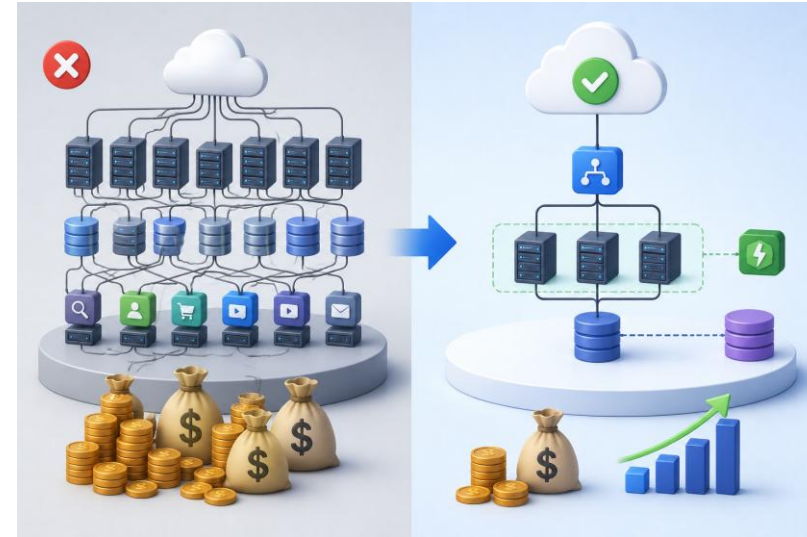
Reliability



Performance



Maintainability



Cost Efficiency



Security

Evolution of System Design

1995–2005

Early Monolithic Web Applications



2005–2010

Scaling Challenges & Distributed Systems



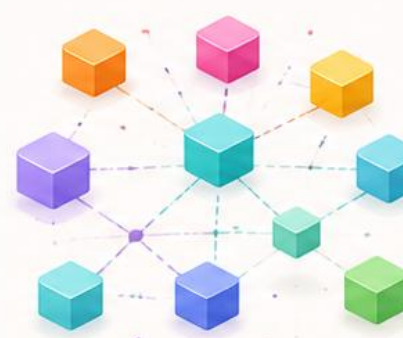
2010–2015

The Cloud Revolution



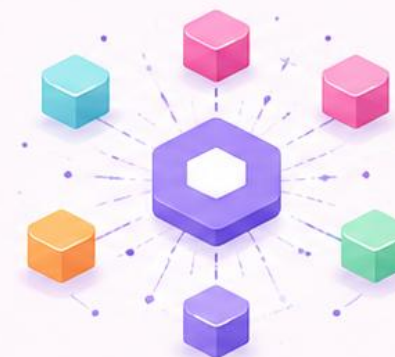
2015–2020

Microservices & Event-Driven Architectures



2020–Present

Real-Time, AI-Driven & Edge Computing



- ✓ Simple to build
- ✓ Everything in one place
- ✓ Easy to understand
- ✗ Hard to scale
- ✗ Tight coupling
- ✗ Single point of failure

- ✓ Better scalability
- ✓ Load distribution
- ✓ Fault tolerance improved
- ✗ Complexity increases
- ✗ Data consistency challenges

- ✓ Elastic infrastructure
- ✓ Pay-as-you-go
- ✓ High availability
- ✓ Managed services
- ✗ Vendor lock-in
- ✗ Cost management needed

- ✓ Independent deployment
- ✓ Technology flexibility
- ✓ Resilient & scalable
- ✓ Event-driven integration
- ✗ Operational complexity
- ✗ Distributed system challenges

- ✓ Real-time processing
- ✓ AI/ML capabilities
- ✓ Edge intelligence
- ✓ Personalized experiences
- ✗ High complexity
- ✗ Privacy & security concerns

4-step Framework for System Design

Step 1: Understand Problem & Scope

Step 2: Estimate Scale & Bottlenecks

Step 3: High-Level Design

Step 4: Tech & Infra Decisions

DESIGNING A NEWS FEED SYSTEM INSTAGRAM

What is a news feed?

A news feed is the core feature of social media platforms like Instagram, Twitter/X, Facebook, and LinkedIn.

It displays posts from users, pages, or accounts that a user follows.

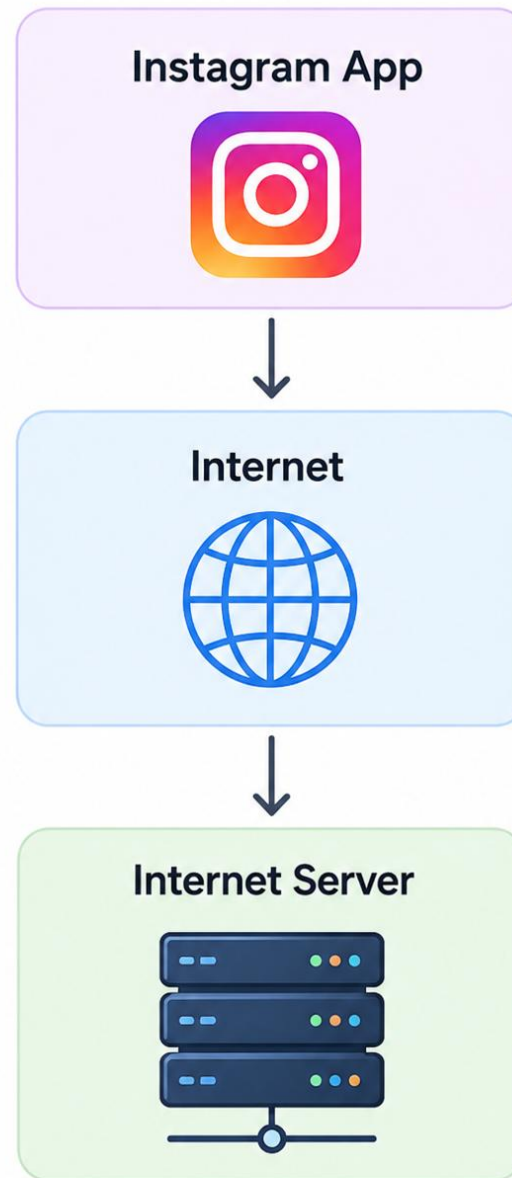
The feed should be personalized, fast, scalable, and highly available.

STEP 1: UNDERSTAND PROBLEM & SCOPE

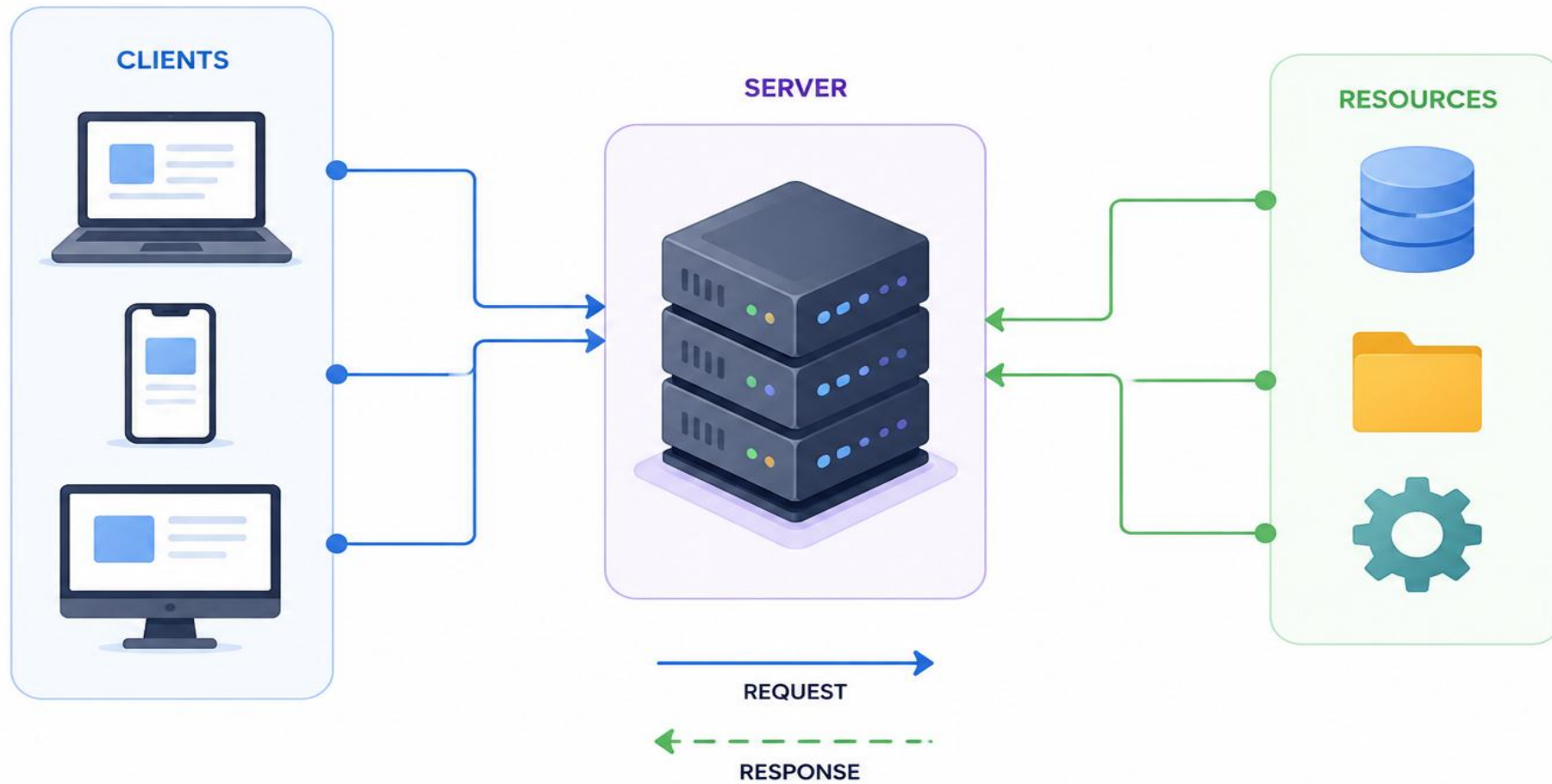
Core Functional Requirements



- Post a Tweet/Update
- Follow/unfollow users
- View Home Timeline
- Timeline must show updates from followed users
- Likes, replies, retweets
- Media uploads(images, videos)
- Interaction Flow



Client-Server Architecture



Client:

The user facing application that sends the request

Examples: Web browser, mobile app, API consumers

Server:

A system that process requests and returns responses

Examples: web server, database server, mail server

Network:

The medium that facilitates communication between clients and servers

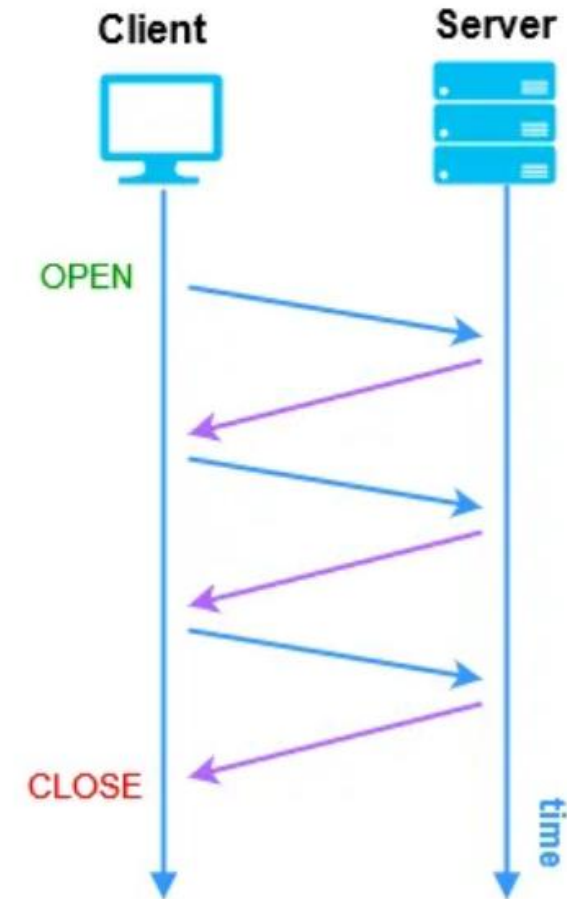
Examples: Internet, LAN, WIFI

Two Types of Connections

Request-Response



Persistent Connections



Synchronous vs Asynchronous

SYNCHRONOUS

Client waits for the response before proceeding.



Client



Client is blocked and waits for the response.

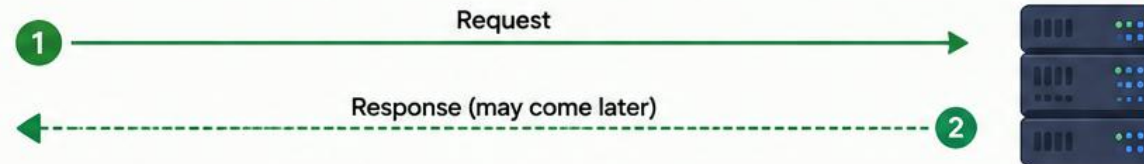


ASYNCHRONOUS

Client doesn't wait for the response and can perform other tasks.



Client



Client is NOT blocked. It can continue other tasks.



Stateless Servers: They do not store any information about previous client interactions. Every request is treated as a completely new and independent request.

This design improves scalability, reliability, and resource efficiency because any server can handle any incoming request.

Stateless servers work exceptionally well with load balancers, as requests can be distributed across multiple servers without needing session-specific routing.

Stateless Server

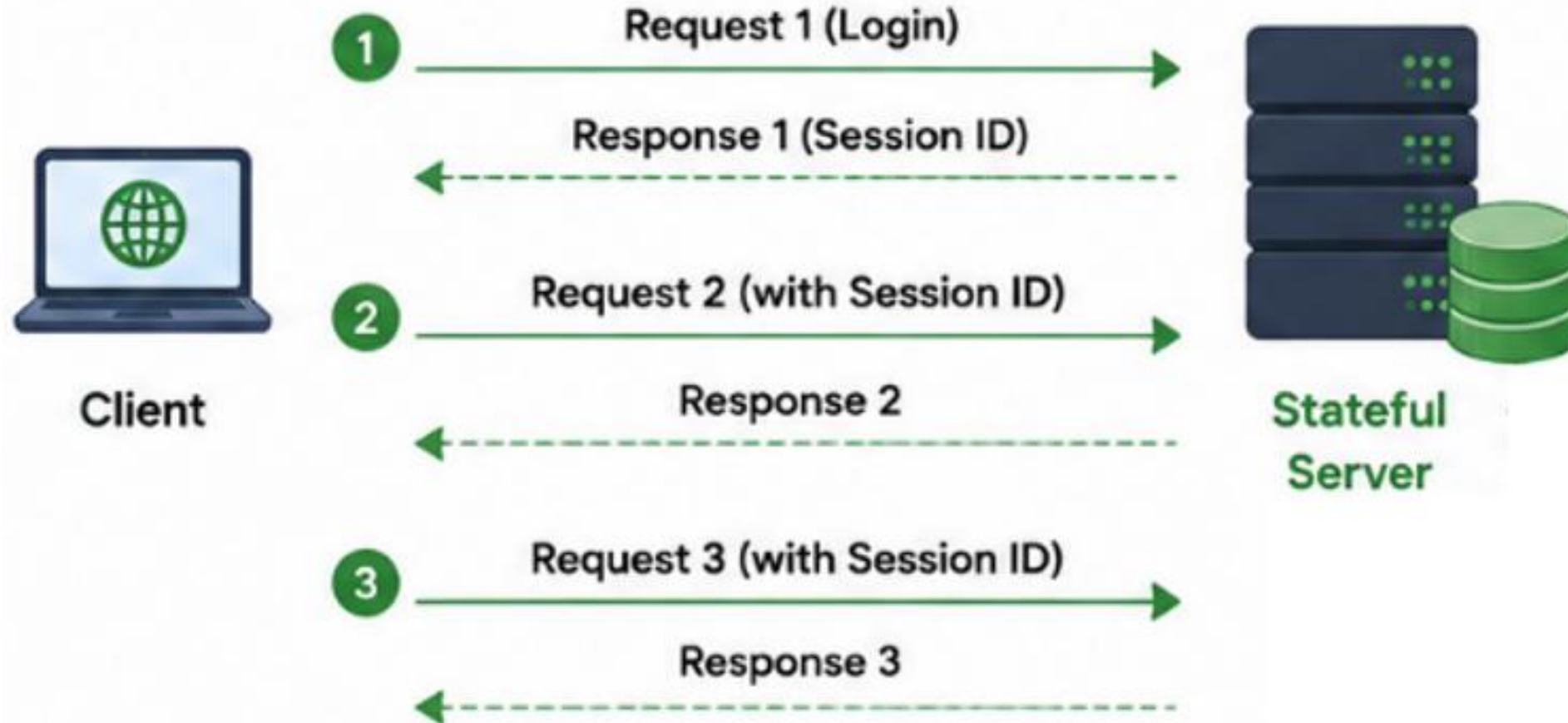


Stateful Servers: They maintain and store session information about users across multiple requests, allowing the server to remember previous interactions.

Benefits: This improves user experience and personalization by preserving user-specific data such as login sessions, shopping carts, preferences, and application state.

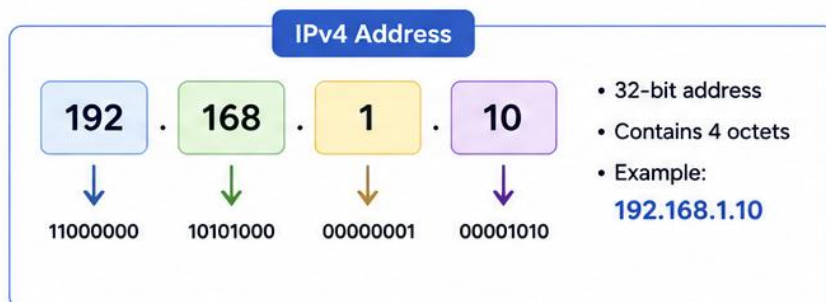
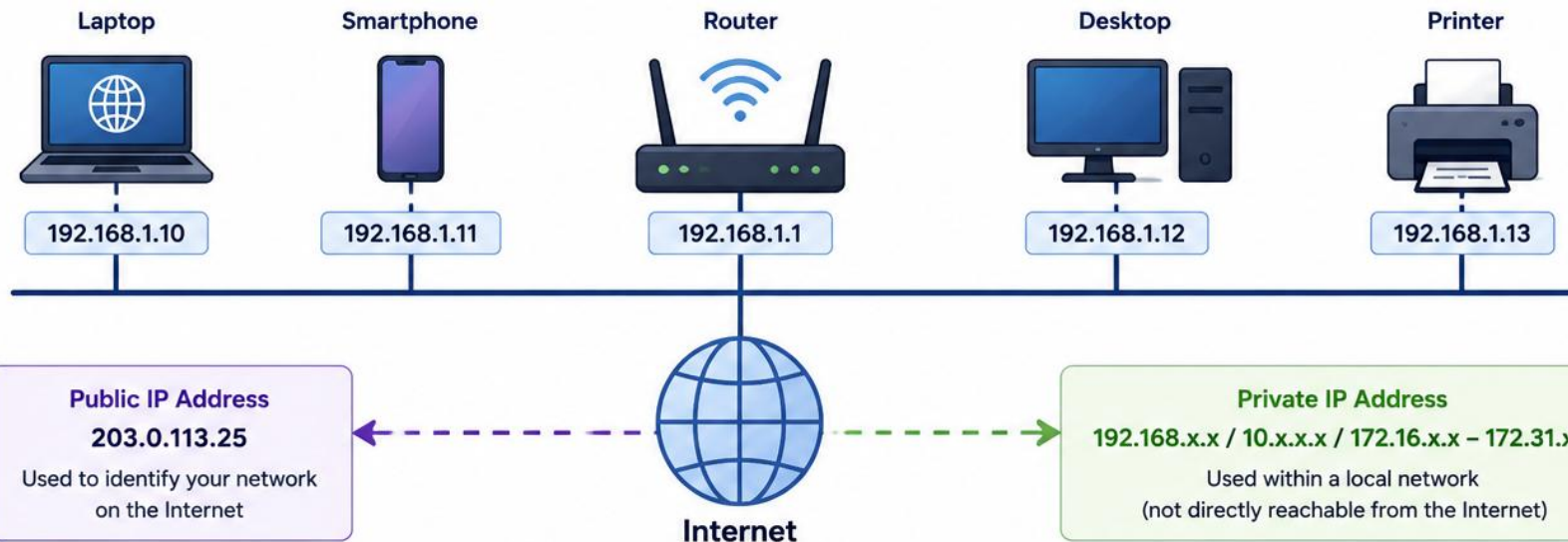
Challenges: Stateful architectures can be more complex to scale because requests from the same user may need to be routed to the same server or rely on shared session storage.

Stateful Server



IP Addresses

Every device connected to a network has a unique IP address that helps it identify and communicate with other devices.



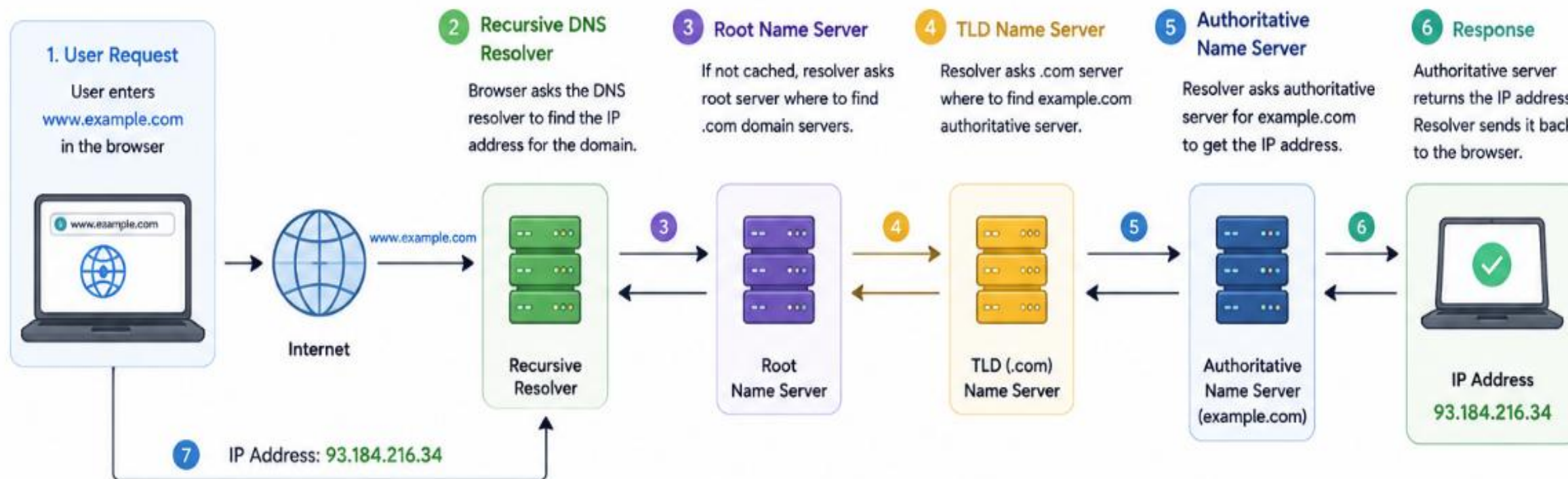
Key Points

- ✓ IP address helps in identifying devices on a network.
- ✓ IPv4 is running out of addresses, IPv6 is the future.
- ✓ Private IPs are used inside local networks.
- ✓ Public IP is used to communicate over the Internet.

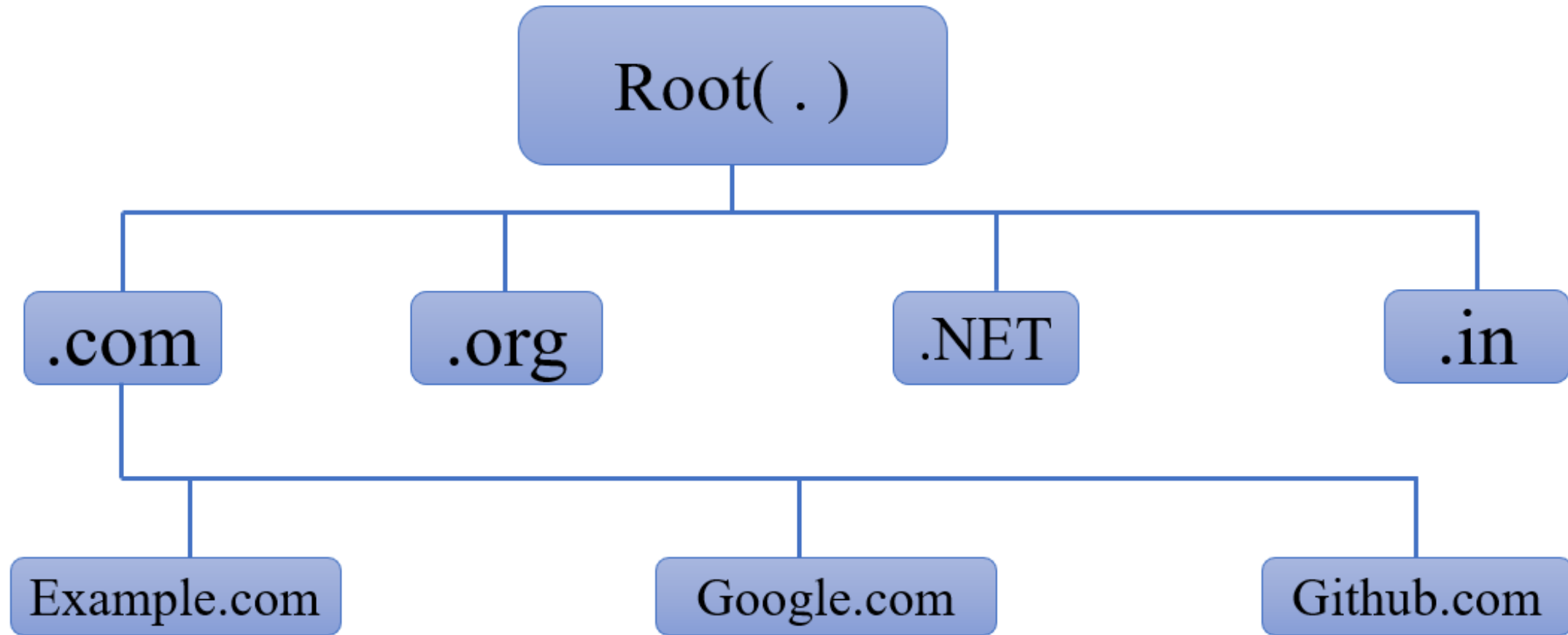
IP Type	Purpose
Private IP	Communication within local network
Public IP	Communication over the Internet

DNS (Domain Name System)

DNS translates human-friendly domain names into IP addresses that computers use to communicate.



DNS Hierarchy



What is HTTP?

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for transmitting data over the web.

Client (Browser)



1. HTTP Request

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome/125.0
Accept: text/html
...
```

2. HTTP Response

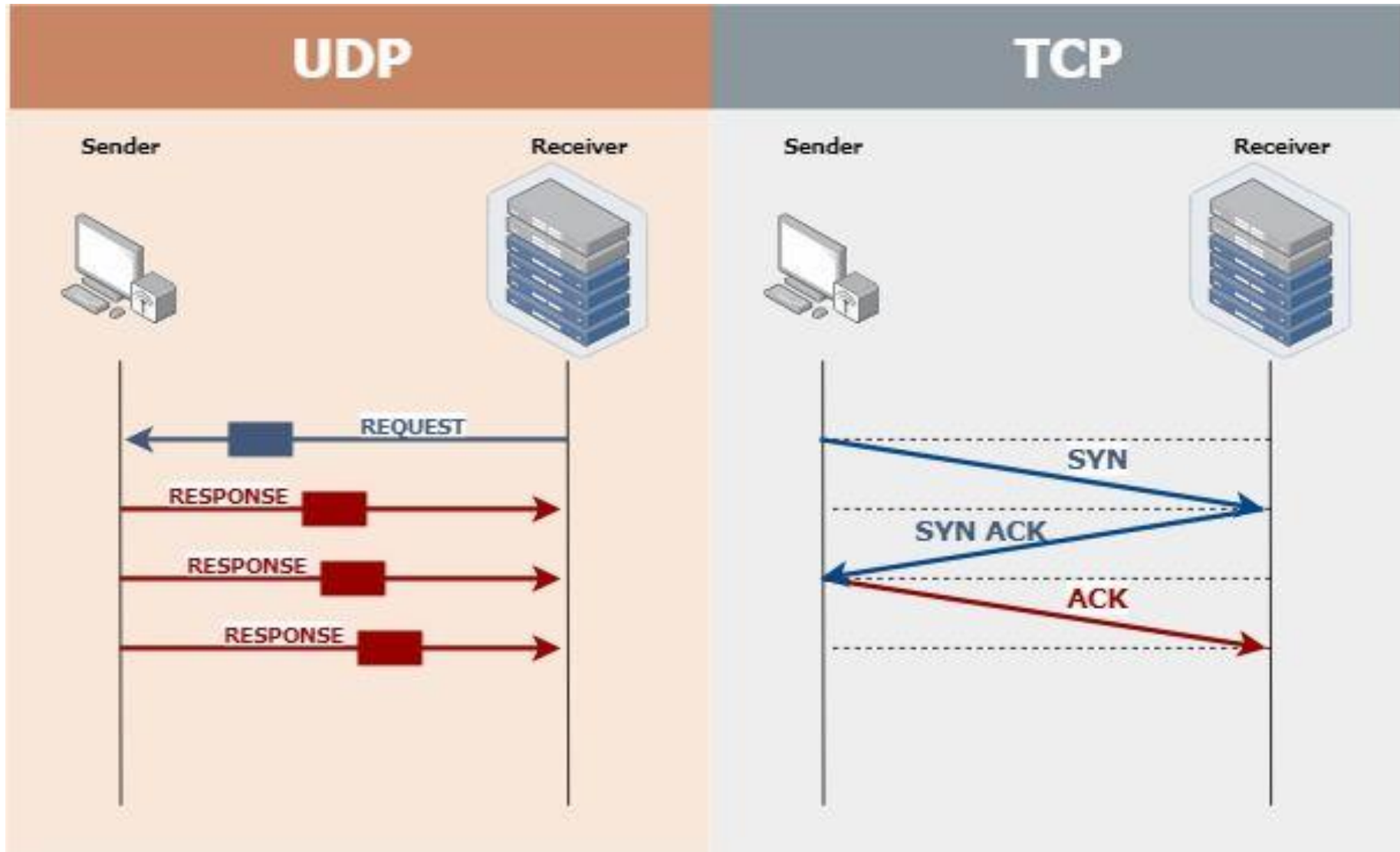
```
HTTP/1.1 200 OK
Content-Type: text/html
Content Length: 1256

<html>
  <body> ... </body>
</html>
```

Server (Web Server)



TCP & UDP



TCP

Connection-Oriented

Guaranteed Transmission

Critical applications

Slower

20 bytes header

Three-Way Handshake

Acknowledgment Mechanism

HTTP, HTTPS, FTP, SMTP, etc.

UDP

Connectionless

No Guarantee

Real-Time applications

Faster

8 bytes header

No Handshake Mechanism

No Acknowledgment

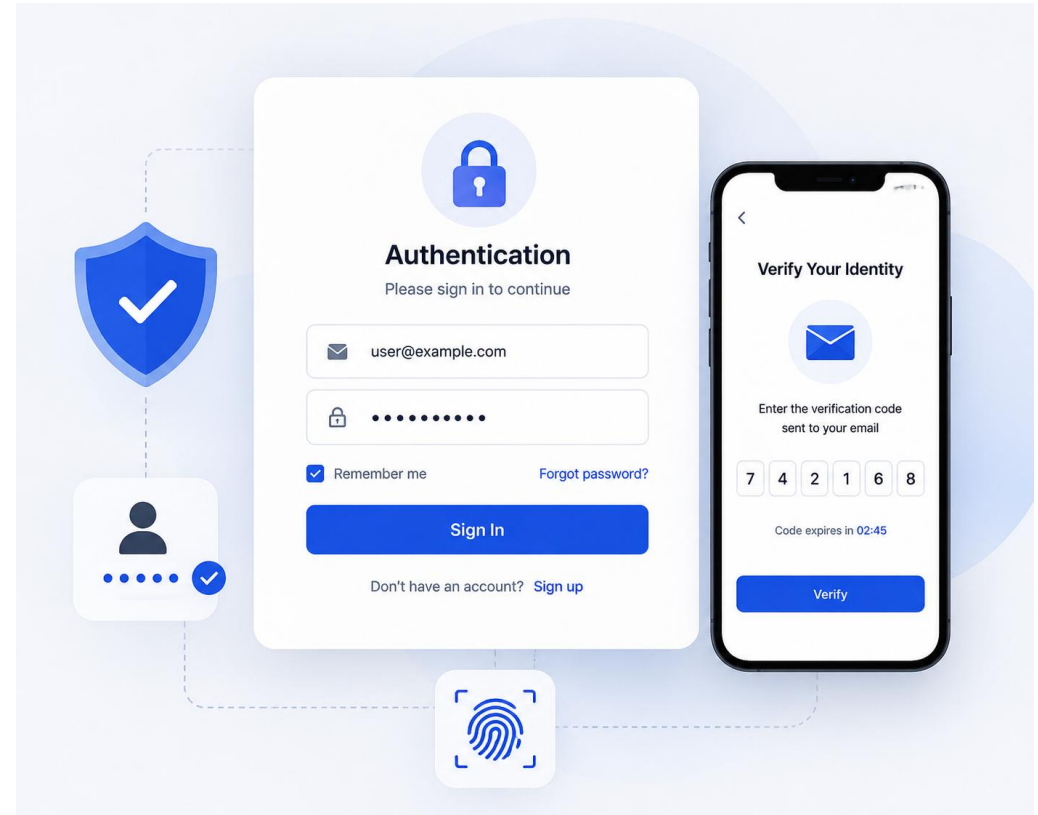
DNS, DHCP, TFTP, SNMP, etc.

Authentication

Authentication is the process of verifying the identity of a user, application, or system.

Authentication helps:

- Protect sensitive data.
- Prevent unauthorized access.
- Secure applications and systems.
- Ensure only valid users can access resources.



Common Authentication Mechanisms

1. Username & Password Authentication
2. OAuth 2.0
3. OpenID Connect (OIDC)
4. JWT (JSON Web Token)

Session Based Authentication

Server-side storage of session data,
typically using cookies

Pros: Easy to implement, works well
with traditional web applications.

Cons: Scalability challenges in
distributed systems

Token Based Authentication

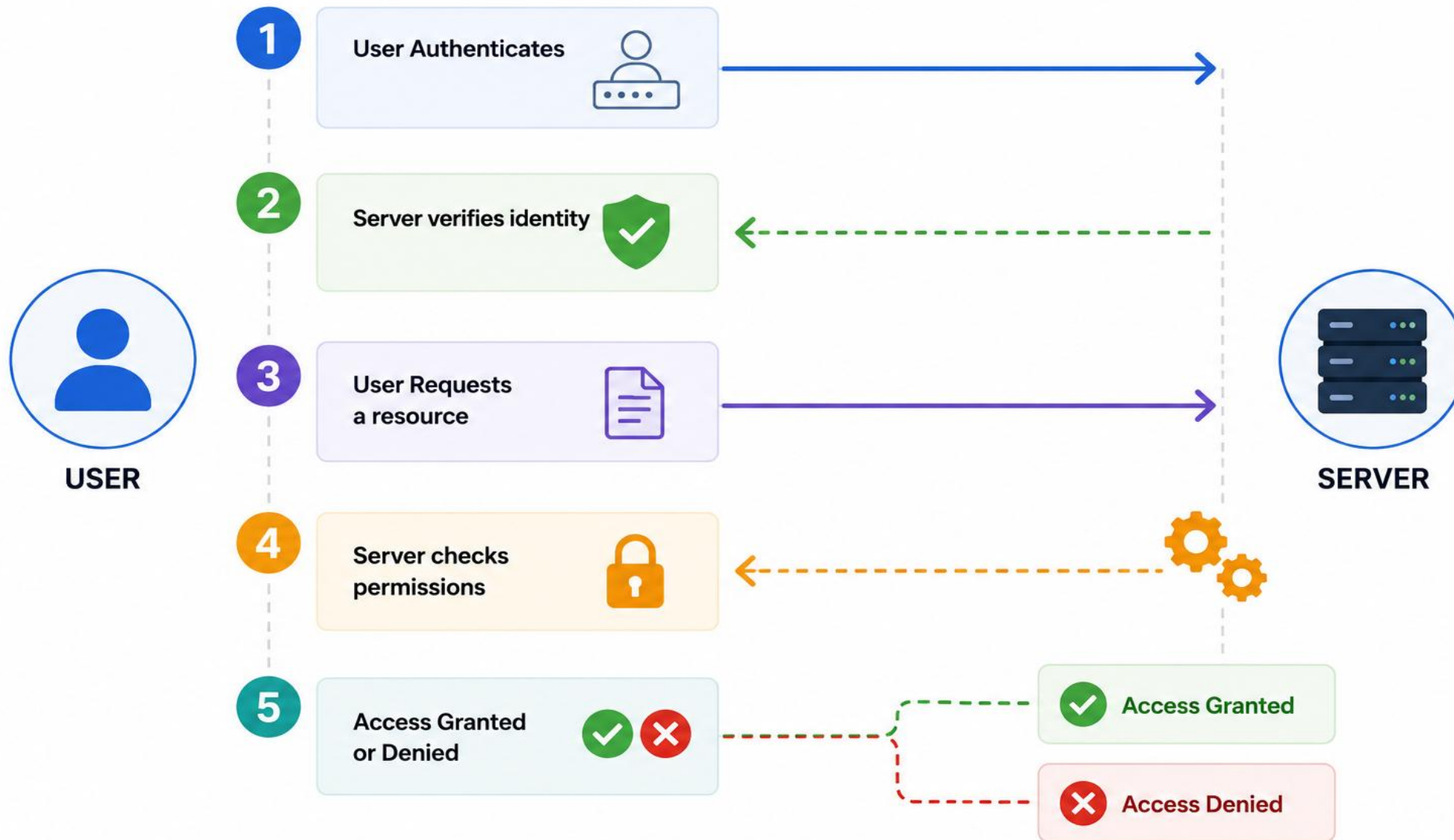
Stateless, where tokens are used to
authenticate the user

Pros: Scalable, decouples backend
system

Cons: Requires secure token
storage and handling

Authorization

Authorization is the process of determining what actions or resources an authenticated user is allowed to access.



Access control models



- RBAC- Role bases Access Control
 - Assigns based on user roles
- ABAC- Attribute –Based Access Control
 - Grants access based on attributes
- DAC- Discretionary Access Control
 - Owner of a resource define access controls
- MAC- Mandatory Access Control
 - Access control decisions are made by a central authority based on security policies

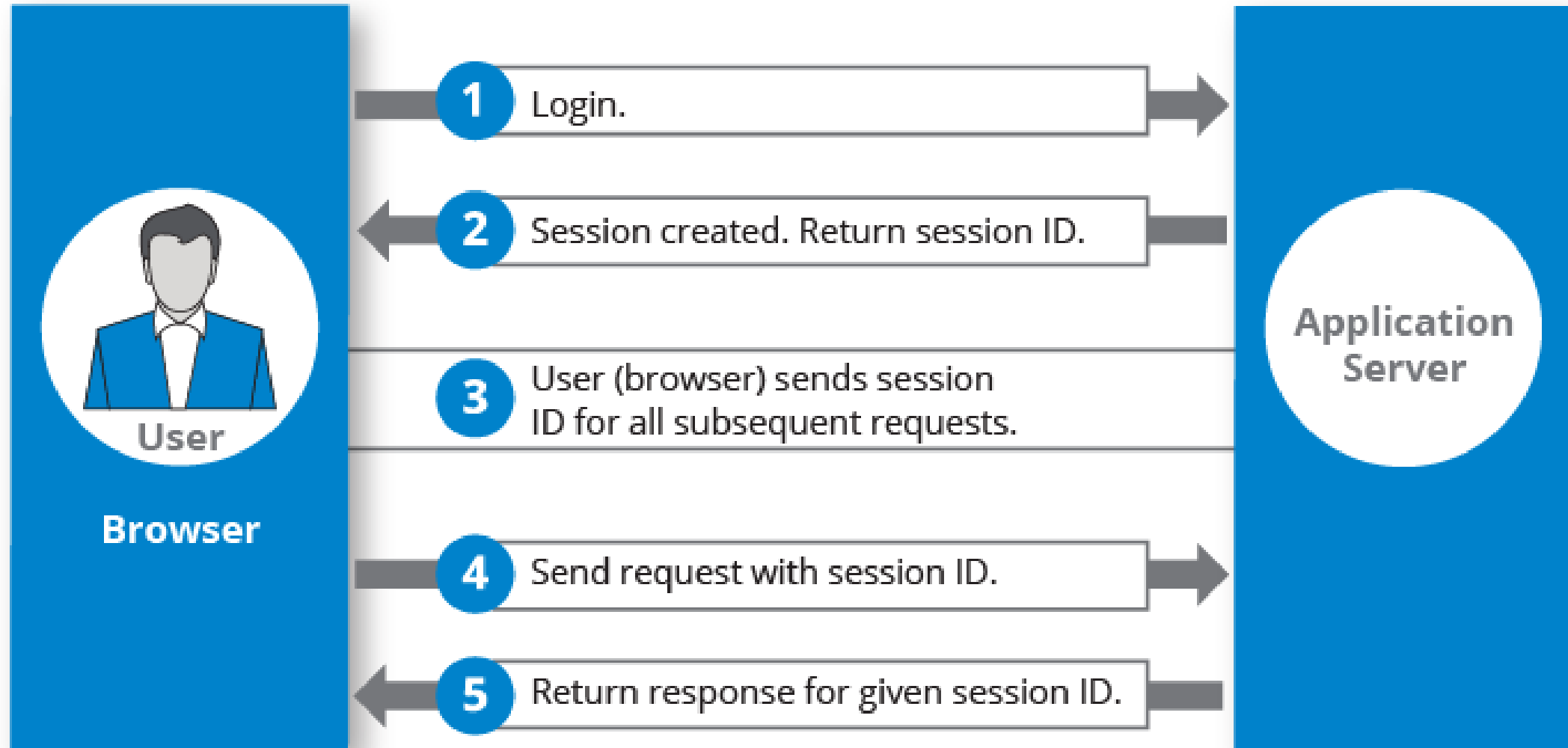
Web Session



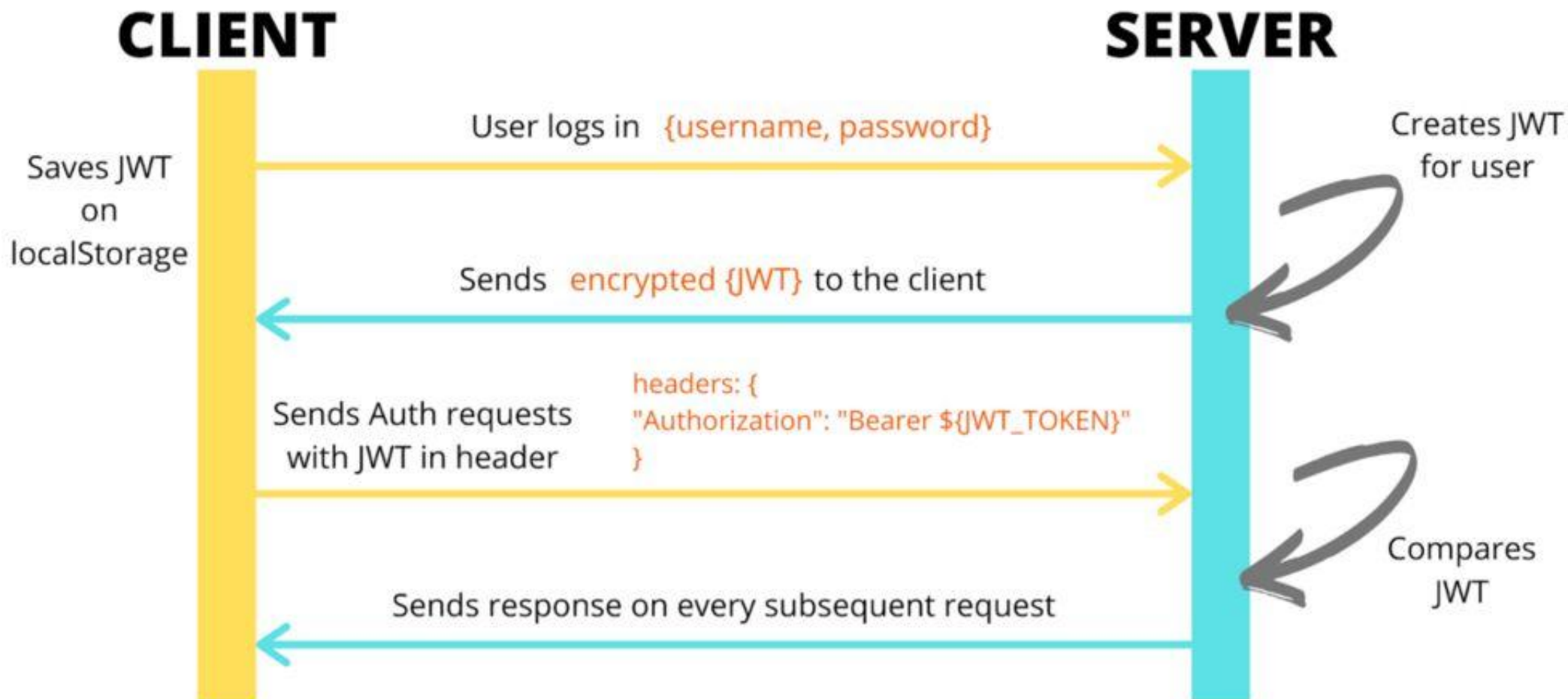
A **Web Session** is a mechanism used to maintain a user's state and information across multiple HTTP requests.

Since **HTTP is a stateless protocol**, the server treats every request as independent. A session allows the server to remember a user between requests.

Session Based



Token Based Authentication



Security Concerns in Session Management



Session Hijacking: Stolen session IDs

Cross-Site Request Forgery: Unauthorized actions

Non-Functional Requirements

- Timeline expectations
- High Availability
- Low Latency
- Scalability

Constraints & Challenges



1. Millions of users
2. Real-time updates
3. Feed must load fast
4. Huge storage growth
5. Read-heavy traffic
6. High availability
7. No duplicate/missing posts
8. Different network speeds
9. Privacy & security
10. Fan-Out challenge